

1. Introduction

This report documents the design, implementation, and testing of a Formula 1 racing database. It includes key entities such as championships, races, teams, drivers, and results. The goal was to create a well-structured relational database that supports complex queries for performance analysis, standings tracking, and event management. I built the database step by step, made changes all around.

This report covers an overall comprehensive overview off the assignment, including:

- **Database design** (ER modeling, relational schema, table descriptions, normalization)
- **Implementation** (tables, constraints, sample data)
- **Query design** (basic and advanced queries for analysis)
- **Advanced features** (procedures, triggers, views)
- **Python integration** (connecting and executing queries programmatically)
- **Reflections** on challenges and lessons learned

When first brainstorming for this project, I took the time to understand how Formula 1 works, including the structure of teams, races, and championships, as I was not aware of it to begin with. This definitely helped me identify the key entities and relationships required for my database. I then examined relevant data sources provided and looked for things beyond and wrote down a schema while considering necessary constraints, like foreign keys. Having a clear design from the beginning helped streamline the implementation process.

2. Database Design

a. ER Modelling

Firstly, entity sets, their keys and attributes:

Entity Set	Key	Other Attributes
Championship	Championship_ID (PK)	Year
Grand Prix	Race_ID (PK)	GP_Name, Date, Location (Country, City, Circuit_Name), Distance
Team	Team_ID (PK)	Name, Country_T, Principal
Driver	Driver_ID (PK)	Name, Nationality, DOB
Ticket	(None)	Average_Price_USD,
Car	Car_ID (PK)	Power_Unit, Chassis, Model_Name

Relationship sets, their participating entity sets and attributes:

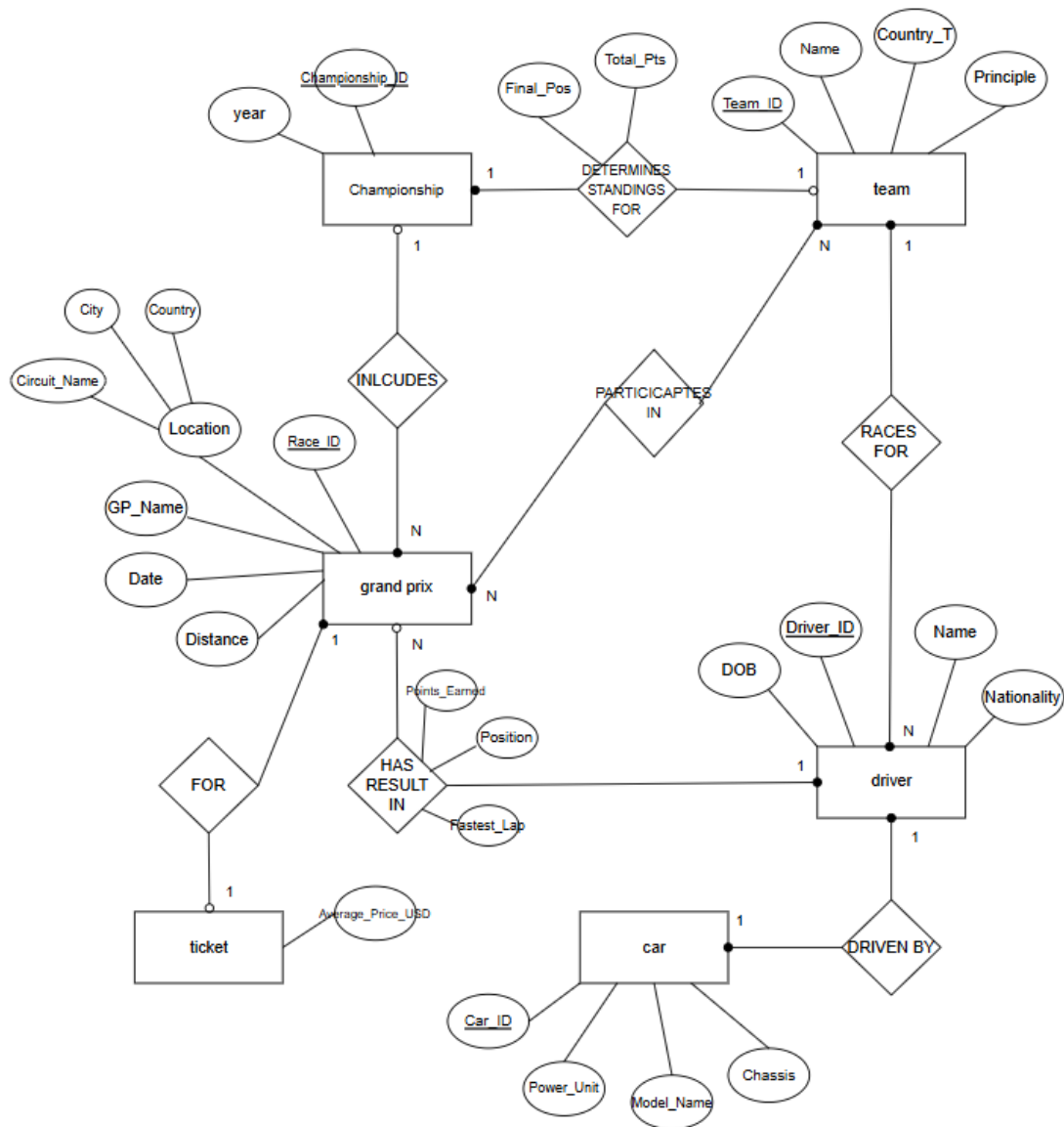
Relationship Set	Between Which Entity Sets	Attributes of Relationship Set (if any)
Includes	Championship, Grand Prix	None
Races for	Driver, Team	None
Participates in	Team, Grand Prix	None

Has a result in	Driver, Grand Prix	Position, Points_Earned, Fastest_Lap
for	Ticket, Grand Prix	None
Determines Standings for	Team, Championship	Final_Position, Total_Points
Driven by	Car, Driver	None

Cardinality and participation constraints

Relationship Set	Cardinality Constraints	Participation/Other Constraints
Includes (Championship, Grand Prix)	One-to-Many (One Championship includes many Grand Prix, each Grand Prix belongs to one Championship)	Mandatory for Grand Prix (every race is part of a championship), Optional for Championship (a championship must have races, but have it without specific races at first)
Races for (Driver, Team)	Many-to-One (Many Drivers race for one Team, but a Driver can only be in one Team at a time)	Mandatory for both (each Driver must belong to a Team, and a Team must have at least one Driver).
Participates in (Team, Grand Prix)	Many-to-Many (A Team participates in many Grand Prix, and each Grand Prix has multiple Teams)	Mandatory for both (Every Team must compete in at least one Grand Prix, and every Grand Prix must have Teams participating).
Has a result in (Driver, Grand Prix)	Many-to-One (A Driver can have only one result per Grand Prix, but each Grand Prix has many Drivers with results)	Mandatory for Driver (Every Driver participating must have a result), Optional for Grand Prix (A Grand Prix can exist before results are recorded)
For (Ticket, Grand Prix)	One-to-One (Each Grand Prix has one recorded average ticket price, and each average ticket price corresponds to one Grand Prix)	Mandatory for Grand Prix (since every race has ticket prices), Optional for Ticket (if data is missing for some races)
Determines Standings for (Team, Championship)	One-to-One (Each Team has one final standing in one Championship)	Mandatory for Championship (every championship has a final team ranking), Optional for Team (if a team didn't participate)
Drives (Driver, Car)	One-to-One (Each Driver has one Car, and each Car is assigned to one Driver)	Mandatory for both (Every Driver must have a Car, and every Car must be assigned to a Driver)

ER Diagram:



b. Define relational schema

- Checked the 1-N relations and added foreign key to the N side from the 1 side
- I decided to merge ticket with Grand Prix table, as it was a 1:1 relation.
- **1:1 relationship** between Team and Championship, but I didn't merge them as its mandatory for Championship but optional for Team, better to keep separate table
- **1:1 relationship** between Driver and Car. Since its mandatory for both, I combined the Car attributes Into the Driver table

RELATIONAL SCHEMA:

Championship (Championship_ID **PK**, Year)

GrandPrix (Race_ID **PK**, GP_Name, Date, Country, City, Circuit_Name, Distance, Average_Price_USD, Championship_ID **FK**)

Team (Team_ID **PK**, Name, Country_T, Principal)

Driver (Driver_ID **PK**, Name, Nationality, DOB, Power_Unit, Chassis, Model_Name, Team_ID **FK**)

ParticipatesIn (Team_ID **PK, FK**, Race_ID **PK, FK**)

DriverResultsGP (Driver_ID **PK, FK**, Race_ID **PK, FK**, Position, Points_Earned, Fastest_Lap)

TeamStandings (Team_ID **PK, FK**, Championship_ID **PK, FK**, Final_Position, Total_Points)

Normalisation check:

1. First Normal Form (1NF)

- First I had to check if all the attributes are atomic, meaning they hold single values for an instance. All attributes seem to be atomic.
- Next I need to make sure every table has a primary key (or attribute that defines each row uniquely). Since I've defined prim keys for all the tables, this condition is met

2. Second Normal form (2NF)

- The goal is to avoid partial dependency, meaning all non-key attributes must depend on the whole primary key, not just a part of it.
- For **ParticipatesIn**, **DriverResultsGP**, and **TeamStandings**, I've made sure that all attributes depend on both keys, so this condition is also met.

3. Third Normal Form (3NF)

- To ensure 3NF, I checked for transitive dependencies. Non-key attributes should depend only on the primary key and not on any other non key attributes (one attribute depends on second, and second on third and so on)
- For example, in the GrandPrix table, all attributes depend only on Race_ID.

Table Descriptions

Championship Table

Description: Stores information about the F1 championships.

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Championship_ID	INT	-	N	Y	Unique ID for each championship.	AUTO_INCREMENT
Name	VARCHAR	100	N	N	Name of the championship	-
Year	INT	-	N	N	Year of the championship.	CHECK (Year >= 1950)

- I added name of championship like F1 as attribute

GrandPrix Table

Description: Stores details about individual Grand Prix races

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Race_ID	INT	-	N	Y	Unique ID for each Grand Prix.	AUTO_INCREMENT
GP_Name	VARCHAR	50	N	N	Name of the Grand Prix.	-
Date	DATE	-	N	N	Date of the Grand Prix.	-
Country	VARCHAR	50	N	N	Country where the race is held.	-
City	VARCHAR	50	N	N	City where the race is held.	-
Circuit_Name	VARCHAR	70	N	N	Name of the racing circuit.	-
Distance	FLOAT	-	Y	N	Total race distance in kilometers.	CHECK (Distance > 0)
Average_Price_USD	DECIMAL	(10,2)	Y	N	Average ticket price for the Grand Prix.	CHECK (Average_Price_USD >= 0)
Championship_ID	INT	-	N	FK	Foreign key referencing Championship.	ON DELETE CASCADE ON UPDATE CASCADE

Team Table

Description: Stores details about F1 teams.

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Team_ID	INT	-	N	Y	Unique ID for each team.	AUTO_INCREMENT
Name	VARCHAR	100	N	UNI	Name of the team.	UNIQUE
Country_T	VARCHAR	50	N	N	Country where the team is based.	-
Principal	VARCHAR	100	Y	N	Team principal's name.	-

Driver Table

Description: Stores details about drivers and their cars.

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Driver_ID	INT	-	N	Y	Unique ID for each driver.	AUTO_INCREMENT
Name	VARCHAR	100	N	N	Full name of the driver.	-
Nationality	VARCHAR	50	N	N	Nationality of the driver.	-
DOB	DATE	-	N	N	Date of birth of the driver.	CHECK (DOB < CURDATE())
Power_Unit	VARCHAR	50	N	N	The engine type used in the car.	-
Chassis	VARCHAR	50	N	N	The chassis model of the car.	-
Model_Name	VARCHAR	50	N	N	The model name of the car.	-
Team_ID	INT	-	Y	FK	Foreign key referencing Team.	ON DELETE SET NULL

ParticipatesIn Table (removed)

Description: Stores information about which teams are participating in which Grand Prix races. This is a many-to-many relationship between the **Team** and **GrandPrix** tables.

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Team_ID	INT	-	N	Y, FK	Foreign key referencing Team.	ON DELETE CASCADE, ON UPDATE CASCADE
Race_ID	INT	-	N	Y, FK	Foreign key referencing GrandPrix.	ON DELETE CASCADE, ON UPDATE CASCADE

DriverResultsGP Table

Description: Stores race results for each driver in a Grand Prix.

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Driver_ID	INT	-	N	Y, FK	Foreign key referencing Driver.	ON DELETE CASCADE

Race_ID	INT	-	N	Y, FK	Foreign key referencing GrandPrix.	ON DELETE CASCADE
Position	INT	-	N	N	Finishing position of the driver.	CHECK (Position > 0)
Points_Earned	DECIMAL	(4,2)	N	N	Points earned by the driver.	CHECK (Points_Earned >= 0)
Fastest_Lap	VARCHAR	-	Y	N	Fastest lap time set by the driver.	-

- Had to update the fastest lap to include milliseconds because that matters, so I changed it to VARCHAR from TIME

TeamStandings Table

Description: Stores the final standings of teams in a championship.

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Team_ID	INT	-	N	Y, FK	Foreign key referencing Team.	ON DELETE CASCADE
Championship_ID	INT	-	N	Y, FK	Foreign key referencing Championship.	ON DELETE CASCADE
Final_Position	INT	-	N	N	The final ranking of the team.	CHECK (Final_Position > 0)
Total_Points	DECIMAL	(6,2)	N	N	Total points earned by the team.	CHECK (Total_Points >= 0)

Business Rules and Assumptions made

(any logic that must be followed in database based on real world requirements)

- Grand Prix must belong to a Championship there are no single races
- Drivers must belong to Team
- Each Grand Prix has an average ticket price which is within the Grand Prix table itself
- Drivers only drive a single car at a time
- Teams can participate in multiple races
- Drivers race results are recorded per Grand Prix, one result per driver per race
- Each team has a final standing position in the championship standings
- Used ONE DELETE CASCADE to ensure automatic deletion of records that are dependent when an FK entity is removed
- Not all race results were included for every Grand Prix due to the large number of driver results for each race. Only a subset of races and their corresponding driver results were added for the sake of simplicity and manageability in the initial implementation.

3. Database implementation & Sampling Data

How I Implemented the database

Evidence of Implementation- Tables created

Screenshots of the tables created in MySQL

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db...
mysql> show tables;
+-----+
| Tables_in_Nazir_21848838 |
+-----+
| Championship              |
| Driver                    |
| DriverResultsGP           |
| GrandPrix                 |
| ParticipatesIn            |
| Team                      |
| TeamStandings             |
+-----+
7 rows in set (0.00 sec)
```

Evidence of Implementation- Inserting data (more was added later on)

A sample SELECT * FROM table output showing inserted data.

Championship table

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernis...
mysql> SELECT * FROM Championship;
+-----+
| Championship_ID | Name | Year |
+-----+
| 1 | F1 | 2024 |
+-----+
1 row in set (0.00 sec)
```

Driver table

Driver_ID	Name	Nationality	DOB	Power_Unit	Chassis
1	Max Verstappen	Netherlands	1997-09-30	Honda RBPT	RB20
2	Lando Norris	United Kingdom	1999-11-13	Mercedes	MCL38
3	Charles Leclerc	Monaco	1997-10-16	Ferrari	SF-24
4	Oscar Piastri	Australia	2001-04-06	Mercedes	MCL38
5	Carlos Sainz	Spain	1994-09-01	Ferrari	SF-24
6	George Russell	United Kingdom	1998-02-15	Mercedes	W15
7	Lewis Hamilton	United Kingdom	1985-01-07	Mercedes	W15
8	Sergio Perez	Mexico	1990-01-26	Honda RBPT	RB20
9	Fernando Alonso	Spain	1981-07-29	Mercedes	AMR24
10	Pierre Gasly	France	1996-02-07	Renault	AS24
11	Nico Hulkenberg	Germany	1987-08-19	Ferrari	VF-24
12	Yuki Tsunoda	Japan	2000-05-11	Honda RBPT	VCARB 01
13	Lance Stroll	Canada	1998-10-29	Mercedes	AMR24
14	Esteban Ocon	France	1996-09-17	Renault	AS24
15	Kevin Magnussen	Denmark	1992-10-05	Ferrari	VF-24
16	Alexander Albon	Thailand	1996-03-23	Mercedes	FW46
17	Daniel Ricciardo	Australia	1989-07-01	Honda RBPT	VCARB 01
18	Oliver Bearman	United Kingdom	2005-05-08	Ferrari	VF-24
19	Franco Colapinto	Argentina	2003-05-27	Mercedes	FW46
20	Guanyu Zhou	China	1999-05-30	Ferrari	C44
21	Liam Lawson	New Zealand	2002-02-11	Honda RBPT	VCARB 01
22	Valtteri Bottas	Finland	1989-08-28	Ferrari	C44
23	Logan Sargeant	United States	2000-12-31	Mercedes	FW46

DriverResultsGP Table

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_DbS2025
```

```
mysql> SELECT * FROM DriverResultsGP limit 10;
```

Driver_ID	Race_ID	Position	Points_Earned	Fastest_Lap
1	1	1	26.00	01:32:608
1	2	1	25.00	01:31:773
1	3	NC	0.00	01:23:115
2	1	6	8.00	01:34:476
2	2	8	4.00	01:31:944
2	3	3	15.00	01:19:915
3	1	4	12.00	01:34:090
3	2	3	16.00	01:31:632
3	3	2	19.00	01:19:813
4	1	8	4.00	01:34:983

10 rows in set (0.00 sec)

GrandPrix Table

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_DbS2025
```

Race_ID	GP_Name	Date	Country	City	Circuit_Name	Distance	Average_Price_USD	Champ
1	Gulf Air Bahrain GP	2024-03-02	Bahrain	Sakhir	Bahrain International Circuit	308.238	265.00	
2	STC Saudi Arabian GP	2024-03-09	Saudi Arabia	Jeddah	Jeddah Corniche Circuit	308.45	391.00	
3	Rolex Australian GP	2024-03-24	Australia	Melbourne	Albert Park Grand Prix Circuit	306.124	290.00	
4	MSC Cruises Japanese GP	2024-04-07	Japan	Suzuka	Suzuka Circuit	307.471	264.00	
5	Lenovo Chinese GP	2024-04-21	China	Shanghai	Shanghai International Circuit	305.066	199.00	

Team Table

```
mysql> SELECT * FROM Team;
```

Team_ID	Name	Country_T	Principal
1	McLaren Mercedes	United Kingdom	Andrea Stella
2	Ferrari	Italy	Frédéric Vasseur
3	Red Bull Racing Honda RBPT	United Kingdom	Christian Horner
4	Mercedes	United Kingdom	Toto Wolff
5	Aston Martin Aramco Mercedes	United Kingdom	Mike Crack
6	Alpine Renault	United Kingdom	Oliver Oakes
7	Haas Ferrari	United States	Ayao Komatsu
8	RB Honda RBPT	Italy	Laurent Mekies
9	Williams Mercedes	United Kingdom	James Vowles
10	Kick Sauber Ferrari	Switzerland	Alessandro Alunni Bravi

10 rows in set (0.00 sec)

TeamStandings Table

```
mysql> SELECT * FROM TeamStandings;
```

Team_ID	Championship_ID	Final_Position	Total_Points
1	1	1	666.00
2	1	2	652.00
3	1	3	589.00
4	1	4	468.00
5	1	5	94.00
6	1	6	65.00
7	1	7	58.00
8	1	8	46.00
9	1	9	17.00
10	1	10	4.00

10 rows in set (0.00 sec)

Description of sample data, data sources and how I inserted data into database

The data was inserted into the database using SQL INSERT statements, with foreign key relationships ensuring consistency between tables. For example, the *TeamStandings* table was populated with final team standings based on actual 2024 results, that I got from the official F1 website.

As I was setting up my database, I realized that the *ParticipatesIn* table, which tracked team participation in each race, was redundant, as most teams participate in every race. I decided to simplify the schema by removing this table, reducing complexity and avoiding unnecessary data redundancy.

Moreover, instead of dynamically calculating the team standings through queries, I manually inserted the final team standings for the 2024 season, aligning with the assignment's focus on past data.

Sources:

I used two sources to populate the tables in my database. I chose the year 2024 F1 and inserted data relevant to that year into my database.

- **Github link (provided on blackboard):**
https://github.com/toUpperCase78/formula1-datasets/blob/master/Formula1_2024season_calendar.csv
- **Official F1 site :** <https://www.formula1.com/en/results/2024/team>

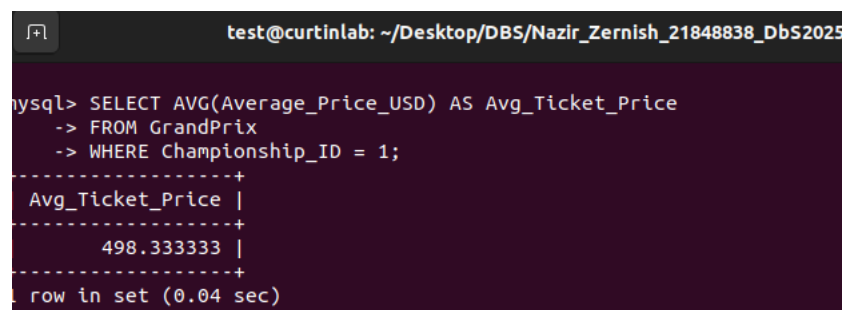
4. Using the Database

Design and implement queries

Level1:

1. Average ticket price for all Grand prix for Championship_ID 1

Question- What is the average ticket price across all Grand Prix events in the F1 Championship 2024 (first ID)



```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_DbS2025
mysql> SELECT AVG(Average_Price_USD) AS Avg_Ticket_Price
-> FROM GrandPrix
-> WHERE Championship_ID = 1;
+-----+
| Avg_Ticket_Price |
+-----+
| 498.333333 |
+-----+
1 row in set (0.04 sec)
```

Purpose: this query can calculate the average ticket price for all Grand Prix races in the first championship (or can be changed to any other championship). Is useful for pricing strategies or understanding how the ticket costs vary. Can be useful to compare with other years, and helps plan future events.

2. Find Races Held in a specific month (March)

Question- Which Grand Prix races are held in the month of ___?

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Dbs2025
mysql>
mysql> SELECT GP_Name, Date
-> FROM GrandPrix
-> WHERE MONTH(Date) = 3;
+-----+-----+
| GP_Name          | Date       |
+-----+-----+
| Gulf Air Bahrain GP | 2024-03-02 |
| STC Saudi Arabian GP | 2024-03-09 |
| Rolex Australian GP  | 2024-03-24 |
+-----+-----+
3 rows in set (0.00 sec)
```

Purpose: Useful for even planning, promotions, and identifying seasonal race trends. What I wish to know by the query is simply the name of the GPs in the specified month.

3. Find Drivers Whose Names Start with 'L' or who are Italian

Question- Which drivers have names starting with 'L' or have Italian nationality?

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Dbs2025
mysql> SELECT Name, Nationality
-> FROM Driver
-> WHERE Name LIKE 'L%' OR Nationality = 'Italy';
+-----+-----+
| Name              | Nationality |
+-----+-----+
| Lando Norris      | United Kingdom |
| Lewis Hamilton    | United Kingdom |
| Lance Stroll      | Canada      |
| Liam Lawson       | New Zealand  |
| Logan Sargeant     | United States |
+-----+-----+
5 rows in set (0.01 sec)
```

Purpose: The query helps analyse drivers based on name patterns and nationality which may be useful for regional marketing, fan engagement, or just overall analysis.

4. Convert Driver Names to Uppercase (With a Date condition)

Question- show all the driver names in uppercase where the driver's DOB is after January 1st, 1995

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db52025

mysql> SELECT UPPER(Name) AS Uppercase_Name
-> FROM Driver
-> WHERE DOB > '1995-01-01';
+-----+
| Uppercase_Name |
+-----+
| MAX VERSTAPPEN |
| LANDO NORRIS   |
| CHARLES LECLERC |
| OSCAR PIASTRI  |
| GEORGE RUSSELL |
| PIERRE GASLY   |
| YUKI TSUNODA   |
| LANCE STROLL   |
| ESTEBAN OCON   |
| ALEXANDER ALBON |
| OLIVER BEARMAN |
| FRANCO COLAPINTO |
| GUANYU ZHOU    |
| LIAM LAWSON    |
| LOGAN SARGEANT |
+-----+
```

Purpose: The query is a demonstration of string manipulation using the upper function while filtering drivers born after 1995. This query could help analyse younger drivers competing in F1, useful query for reports and formatting. Helps highlight more recent drivers in a standardised format.

5. Extracts the Year from Driver's DOB

Question- What is the birth year of each driver?

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db52025

mysql> SELECT Name, YEAR(DOB) AS Birth_Year
-> FROM Driver
-> ORDER BY DOB;
+-----+
| Name           | Birth_Year |
+-----+
| Fernando Alonso | 1981       |
| Lewis Hamilton  | 1985       |
| Nico Hulkenberg | 1987       |
| Daniel Ricciardo | 1989       |
| Valtteri Bottas | 1989       |
| Sergio Perez    | 1990       |
| Kevin Magnussen | 1992       |
| Carlos Sainz    | 1994       |
| Pierre Gasly    | 1996       |
| Alexander Albon | 1996       |
| Esteban Ocon    | 1996       |
| Max Verstappen  | 1997       |
| Charles Leclerc | 1997       |
| George Russell  | 1998       |
| Lance Stroll    | 1998       |
| Guanyu Zhou     | 1999       |
| Lando Norris    | 1999       |
| Yuki Tsunoda    | 2000       |
| Logan Sargeant  | 2000       |
| Oscar Piastri   | 2001       |
| Liam Lawson     | 2002       |
| Franco Colapinto | 2003       |
| Oliver Bearman  | 2005       |
+-----+
23 rows in set (0.00 sec)
```

Purpose: Extracts the year of birth for each driver and orders the result by DOB, which can be helpful in age-based performance analysis, generational comparisons, and overall identifying youngest/oldest drivers in the sport. Also, easy chronological analysis

Design and implement queries Level2:

Briefly describe what you wish to know by using the query, why

1. Find the total points earned by each driver across all races

Question- How many points has each driver accumulated across all Grand Prix races?

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db...
mysql> SELECT d.Name AS Driver_Name, SUM(dr.Points_Earned) AS Total_Points
-> FROM DriverResultsGP dr
-> JOIN Driver d ON dr.Driver_ID = d.Driver_ID
-> GROUP BY d.Name
-> ORDER BY Total_Points DESC;
+-----+-----+
| Driver_Name | Total_Points |
+-----+-----+
| Max Verstappen | 102.00 |
| Sergio Perez | 79.00 |
| Rhythmbox eclerc | 71.00 |
| Carlos Sainz | 65.00 |
| Lando Norris | 55.00 |
| Oscar Piastri | 36.00 |
| George Russell | 32.00 |
| Fernando Alonso | 31.00 |
| Lewis Hamilton | 12.00 |
| Lance Stroll | 9.00 |
| Yuki Tsunoda | 7.00 |
| Oliver Bearman | 6.00 |
| Nico Hulkenberg | 4.00 |
| Kevin Magnussen | 1.00 |
| Esteban Ocon | 0.00 |
| Alexander Albon | 0.00 |
| Daniel Ricciardo | 0.00 |
| Pierre Gasly | 0.00 |
| Guanyu Zhou | 0.00 |
| Valtteri Bottas | 0.00 |
| Logan Sargeant | 0.00 |
+-----+-----+
```

Purpose: The query helps analyse each drivers performance by calculating their total points. Its useful for determining overall rankings and evaluating their consistency across races

Structure: I used an INNER JOIN between the *DriverResultGP* and *Driver* to get each drivers name along with their total points. I used INNER JOIN since every result in *DriverResultsGP* must correspond to an existing driver. The SUM() function totals points per driver and I used GROUP BY *d.Name* ensures we calculate totals per driver. The ORDER BY *Total_Points* DESC sorts drivers from highest to lowest points.

2. Find the winning driver and fastest lap for each gran Prix

Question- Who won each Grand Prix, and what was the fastest lap time in each case?

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db...
mysql> SELECT g.GP_Name, d.Name AS Winner, dr.Fastest_Lap
-> FROM DriverResultsGP dr
-> JOIN Driver d ON dr.Driver_ID = d.Driver_ID
-> JOIN GrandPrix g ON dr.Race_ID = g.Race_ID
-> WHERE dr.Position = 1
-> ORDER BY g.Date;
+-----+-----+-----+
| GP_Name | Winner | Fastest_Lap |
+-----+-----+-----+
| Gulf Air Bahrain GP | Max Verstappen | 01:32:608 |
| STC Saudi Arabian GP | Max Verstappen | 01:31:773 |
| Rolex Australian GP | Carlos Sainz | 01:20:031 |
| MSC Cruises Japanese GP | Max Verstappen | 01:33.706 |
| Lenovo Chinese GP | Max Verstappen | 01:38.406 |
+-----+-----+-----+
5 rows in set (0.01 sec)

mysql>
```

Purpose: The query will provide a clearer summary of each race, showing the winning driver and their fastest lap. Its useful for identifying performances that were better and race highlights.

Structure: I one INNER JOIN between *DriverResultsGP* and *Driver* to match each result to the driver, and another between *DriverResultsGP* and *GrandPrix* to retrieve the Grand Prix name. Since I only wanted the winners, I specified WHERE to be 1st position and added ORDER BY date so it sorted the races chronologically.

3. Find the Fastest Lap for each grand Prix

Question- What was the fastest lap time recorded in each Grand Prix?

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db...
mysql> SELECT g.GP_Name, MIN(dr.Fastest_Lap) AS Fastest_Lap_Time
-> FROM DriverResultsGP dr
-> JOIN GrandPrix g ON dr.Race_ID = g.Race_ID
-> GROUP BY g.GP_Name
-> ORDER BY Fastest_Lap_Time;
+-----+-----+
| GP_Name          | Fastest_Lap_Time |
+-----+-----+
| Rolex Australian GP | 01:19:813        |
| STC Saudi Arabian GP | 01:31:632        |
| Gulf Air Bahrain GP | 01:32:608        |
| MSC Cruises Japanese GP | 01:33.706      |
| Lenovo Chinese GP  | 01:37.810        |
+-----+-----+
5 rows in set (0.00 sec)
```

Purpose: This query helps track race records by showing the fastest lap time per race. Its valuable information for performance analysis and identifying potential fastest drivers overall

Structure: Joins *DriverResultsGP* with *GrandPrix* to align the lap times with the races. To get the fastest lap per Grand Prix, I used MIN function on fastest lap which finds the lowest lap time for each race. GROUP BY *g.GP_Name* ensures the aggregation happens per race and I ORDER BY *Fastest_Lap_Time* from fastest to slowest lap.

4. Find drivers who have came in top 3 more than once

Question- Which drivers have finished in the top 3 more than once?

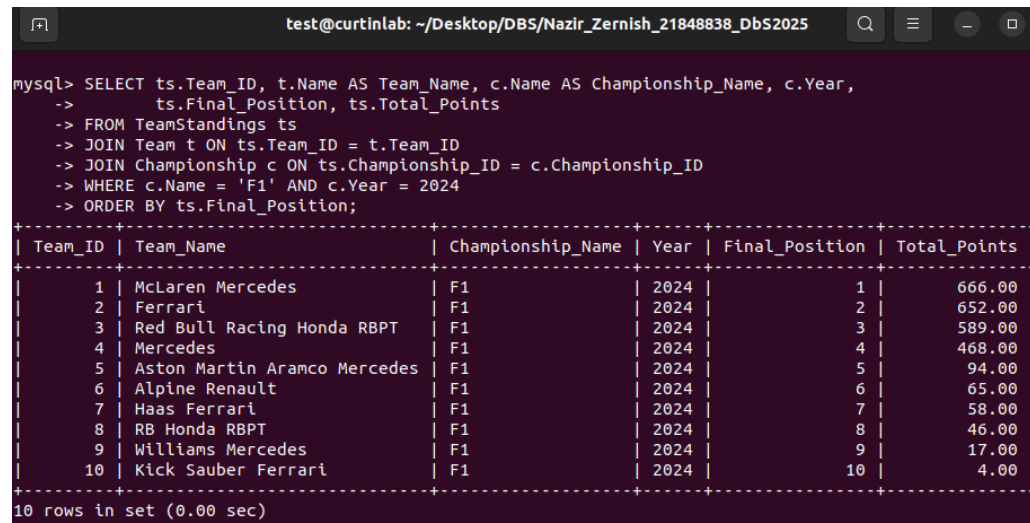
```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db...
mysql> SELECT d.Name AS Driver_Name, COUNT(dr.Position) AS Top_3_Finishes
-> FROM DriverResultsGP dr
-> JOIN Driver d ON dr.Driver_ID = d.Driver_ID
-> WHERE dr.Position <= 3
-> GROUP BY d.Name
-> HAVING COUNT(dr.Position) > 1
-> ORDER BY Top_3_Finishes DESC;
+-----+-----+
| Driver_Name      | Top_3_Finishes |
+-----+-----+
| Max Verstappen   | 5              |
| Sergio Perez     | 4              |
| Carlos Sainz     | 3              |
| Lando Norris     | 2              |
| Charles Leclerc  | 2              |
| Daniel Ricciardo | 2              |
+-----+-----+
6 rows in set (0.00 sec)
```

Purpose: It highlights consistent top performers by counting how many times each driver finished in the top 3. Could help evaluate driver skill and competitiveness too.

Structure: I used an INNER JOIN between *DriverResultsGP* and *Driver* to retrieve driver names alongside their finishing positions. The where condition filters top 3 finishes, and I used COUNT to count how many times each driver finished in the top 3. HAVING COUNT(dr.Position) > 1 made sure it only show drivers who achieved this multiple times, while ORDER BY DESC ranks them accordingly.

5. Clear view of team standings table

Question- What are the final standings of each team in the F1 2024 Championship?



```
mysql> SELECT ts.Team_ID, t.Name AS Team_Name, c.Name AS Championship_Name, c.Year,
->      ts.Final_Position, ts.Total_Points
-> FROM TeamStandings ts
-> JOIN Team t ON ts.Team_ID = t.Team_ID
-> JOIN Championship c ON ts.Championship_ID = c.Championship_ID
-> WHERE c.Name = 'F1' AND c.Year = 2024
-> ORDER BY ts.Final_Position;
```

Team_ID	Team_Name	Championship_Name	Year	Final_Position	Total_Points
1	McLaren Mercedes	F1	2024	1	666.00
2	Ferrari	F1	2024	2	652.00
3	Red Bull Racing Honda RBPT	F1	2024	3	589.00
4	Mercedes	F1	2024	4	468.00
5	Aston Martin Aramco Mercedes	F1	2024	5	94.00
6	Alpine Renault	F1	2024	6	65.00
7	Haas Ferrari	F1	2024	7	58.00
8	RB Honda RBPT	F1	2024	8	46.00
9	Williams Mercedes	F1	2024	9	17.00
10	Kick Sauber Ferrari	F1	2024	10	4.00

10 rows in set (0.00 sec)

Purpose: The TeamStandings table mainly uses IDs since they are primary keys, making it difficult to interpret. This query provides a clearer output by displaying team names, the championship name, and total points.

Structure: join *TeamStandings* with *Team* and *Championship* to provide meaningful team standings instead of just IDs. INNER JOIN used since every team in standings table must have a corresponding entry in Team and Championship. The WHERE clause filters results to only show F1's 2024 standings (in this case), while ORDER BY ts.Final_Position ensures teams are displayed in ranking order.

Design and implementation of advanced features

I implemented:

- two stored procedures: with parameters, variables, IF-ELSE, and loops
- two triggers: BEFORE INSERT and AFTER UPDATE
- one complex view: involving many tables

Stored Procedures

PROCEDURE 1: Calculate Average finishing position of a Driver

Question- What is the average finishing position of a given driver across all races?

```

test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_DbS2025
mysql> DELIMITER //
mysql> CREATE PROCEDURE Get_Avg_Finishing_Position(IN driver_name VARCHAR(100))
-> BEGIN
->     DECLARE avg_position DECIMAL(5,2);
->
->     SELECT AVG(CAST(Position AS UNSIGNED)) INTO avg_position
->     FROM DriverResultsGP drg
->     JOIN Driver d ON drg.Driver_ID = d.Driver_ID
->     WHERE d.Name = driver_name;
->
->     SELECT driver_name AS Driver, avg_position AS Avg_Finishing_Position;
-> END //
Query OK, 0 rows affected (0.22 sec)

mysql> DELIMITER ;
mysql> CALL Get_Avg_Finishing_Position('Max Verstappen');
+-----+-----+
| Driver | Avg_Finishing_Position |
+-----+-----+
| Max Verstappen | 0.80 |
+-----+-----+
1 row in set (0.07 sec)

Query OK, 0 rows affected (0.07 sec)

```

Purpose: this procedure when called helps analyse a drivers' consistency. If their average is low, they are likely a strong competitor, gives an overview.

Structure: the procedure uses a JOIN to link *DriverResultsGP* with *Driver* to match the driver's name. The AVG() function calculates the mean finishing position. Using IN parameters, I can input any driver name dynamically.

PROCEDURE 2: Get races where a driver scored more than X points

Question: In which races did a driver score more than a given number of points?

```

test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db...
mysql> DELIMITER //
mysql> CREATE PROCEDURE Get_High_Scoring_Races(IN driver_name VARCHAR(100), IN min_points DECIMAL(4,2))
-> BEGIN
->     SELECT gp.GP_Name, drg.Points_Earned
->     FROM DriverResultsGP drg
->     JOIN Driver d ON drg.Driver_ID = d.Driver_ID
->     JOIN GrandPrix gp ON drg.Race_ID = gp.Race_ID
->     WHERE d.Name = driver_name AND drg.Points_Earned > min_points
->     ORDER BY drg.Points_Earned DESC;
-> END //
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER ;
mysql> -- Test it:
mysql> CALL Get_High_Scoring_Races('Max Verstappen', 15);
+-----+-----+
| GP_Name | Points_Earned |
+-----+-----+
| Gulf Air Bahrain GP | 26.00 |
| MSC Cruises Japanese GP | 26.00 |
| STC Saudi Arabian GP | 25.00 |
| Lenovo Chinese GP | 25.00 |
+-----+-----+
4 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

```

Purpose: Helps analyse high scoring performances, or just scores above specified point number across all races

Structure: This uses JOINS to link the *DriverResultsGP*, *Driver*, and *GrandPrix* tables. The WHERE clause filters the results based on the drivers name and the point threshold specified. The result is ordered by points in descending order to show the highest scoring races first.

Triggers

TRIGGER 1: Automatically set a default fastest lap when inserting a new race result

Type of trigger: BEFORE INSERT

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Db...
mysql> -- 1. Set a Default Fastest Lap When Inserting a New Race Result
mysql> DELIMITER //
mysql> CREATE TRIGGER Set_Default_Fastest_Lap
-> BEFORE INSERT ON DriverResultsGP
-> FOR EACH ROW
-> BEGIN
->     IF NEW.Fastest_Lap IS NULL THEN
->         SET NEW.Fastest_Lap = '01:30.000';
->     END IF;
-> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> INSERT INTO DriverResultsGP (Driver_ID, Race_ID, Position, Points_Earned)
-> VALUES (5, 12, '3', 15);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM DriverResultsGP WHERE Race_ID = 12;
+-----+-----+-----+-----+-----+
| Driver_ID | Race_ID | Position | Points_Earned | Fastest_Lap |
+-----+-----+-----+-----+-----+
|         5 |        12 | 3       |          15.00 | 01:30.000   |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Purpose: Ensures that if no fastest lap time is provided when inserting a race result, it defaults to 1:30:000 (1 min 30 sec).

Structure: The BEFORE INSERT trigger runs before inserting a new record into *DriverResultsGP*. If *Fastest_Lap* is NULL, it automatically sets it to 01:30.000. This prevents NULL values and ensures data consistency.

TRIGGER 2: Update the team points when a new driver result is inserted

Type of trigger: AFTER INSERT

```

test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_Dbs2025
mysql> SELECT ts.Team_ID, t.Name, ts.Championship_ID, ts.Total_Points
-> FROM TeamStandings ts
-> JOIN Team t ON ts.Team_ID = t.Team_ID
-> JOIN Driver d ON d.Team_ID = t.Team_ID
-> WHERE d.Name = 'Lando Norris'
-> AND ts.Championship_ID = 1; -- 2024 Championship
+-----+-----+-----+-----+
| Team_ID | Name           | Championship_ID | Total_Points |
+-----+-----+-----+-----+
| 1       | McLaren Mercedes | 1              | 666.00       |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> -- Insert a new result for Lando Norris in the Canadian GP (Race_ID 9)
LibreOffice Writer TO DriverResultsGP (Driver_ID, Race_ID, Position, Points_Earned, Fastest_Lap)
-> VALUES (
-> (SELECT Driver_ID FROM Driver WHERE Name = 'Lando Norris'),
-> 9, -- Canadian GP Race_ID
-> '2', -- Position
-> 18, -- Points for 2nd place
-> '01:15.123' -- Sample fastest lap time
-> );
Query OK, 1 row affected (0.03 sec)

mysql> -- Check team standings again to see the update
mysql> SELECT ts.Team_ID, t.Name, ts.Championship_ID, ts.Total_Points
-> FROM TeamStandings ts
-> JOIN Team t ON ts.Team_ID = t.Team_ID
-> JOIN Driver d ON d.Team_ID = t.Team_ID
-> WHERE d.Name = 'Lando Norris'
-> AND ts.Championship_ID = 1;
+-----+-----+-----+-----+
| Team_ID | Name           | Championship_ID | Total_Points |
+-----+-----+-----+-----+
| 1       | McLaren Mercedes | 1              | 684.00       |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

Purpose: Whenever a new result is added to the *DriverResultsGP* table, the total points of the team that the driver belongs to should be updated in the *TeamStandings* table. This ensures that the team's total points correctly reflect the cumulative points earned by its drivers throughout the season.

Structure: The trigger ensures that the driver belongs to a team before updating team standings. It finds the *Team_ID* of the driver who got a new result, adds the newly earned points to the team's existing total points in *TeamStandings*, and finally ensures that the update happens for the correct championship by matching *Championship_ID* in *GrandPrix*

Views

VIEW: Display detailed race results with Driver, team and Grand Prix Info

Question: Can I see a detailed summary of race results, including driver, team, and race info?

GP_Name	Driver	Team	Position	Points_Earned	Fastest_Lap
Qatar Airways British GP	Carlos Sainz	Ferrari	3	15.00	01:30.000
AWS Grand Prix du Canada	Lando Norris	McLaren Mercedes	2	18.00	01:15.123
Lenovo Chinese GP	Valtteri Bottas	Kick Sauber Ferrari	NC	0.00	01:41.276
Lenovo Chinese GP	Yuki Tsunoda	RB Honda RBPT	NC	0.00	01:41.593
Lenovo Chinese GP	Daniel Ricciardo	RB Honda RBPT	NC	0.00	01:40.994
Lenovo Chinese GP	Max Verstappen	Red Bull Racing Honda RBPT	1	25.00	01:38.406
Lenovo Chinese GP	Lando Norris	McLaren Mercedes	2	18.00	01:38.751
Lenovo Chinese GP	Sergio Perez	Red Bull Racing Honda RBPT	3	15.00	01:39.388
Lenovo Chinese GP	Charles Leclerc	Ferrari	4	12.00	01:39.384
Lenovo Chinese GP	Carlos Sainz	Ferrari	5	10.00	01:39.704
Lenovo Chinese GP	George Russell	Mercedes	6	8.00	01:40.112
LibreOfficeWriter GP	Fernando Alonso	Aston Martin Aramco Mercedes	7	7.00	01:37.810
Lenovo Chinese GP	Oscar Piastri	McLaren Mercedes	8	4.00	01:39.739
Lenovo Chinese GP	Lewis Hamilton	Mercedes	9	2.00	01:40.835
Lenovo Chinese GP	Nico Hulkenberg	Haas Ferrari	10	1.00	01:40.815
Lenovo Chinese GP	Esteban Ocon	Alpine Renault	11	0.00	01:40.937
Lenovo Chinese GP	Alexander Albon	Williams Mercedes	12	0.00	01:40.790
Lenovo Chinese GP	Pierre Gasly	Alpine Renault	13	0.00	01:39.198
Lenovo Chinese GP	Guanyu Zhou	Kick Sauber Ferrari	14	0.00	01:38.633
Lenovo Chinese GP	Lance Stroll	Aston Martin Aramco Mercedes	15	0.00	01:39.444
Lenovo Chinese GP	Kevin Magnussen	Haas Ferrari	16	0.00	01:41.077
Lenovo Chinese GP	Logan Sargeant	Williams Mercedes	17	0.00	01:41.000
MSC Cruises Japanese GP	Guanyu Zhou	Kick Sauber Ferrari	NC	0.00	01:37.160
MSC Cruises Japanese GP	Daniel Ricciardo	RB Honda RBPT	NC	0.00	NULL
MSC Cruises Japanese GP	Alexander Albon	Williams Mercedes	NC	0.00	NULL
MSC Cruises Japanese GP	Max Verstappen	Red Bull Racing Honda RBPT	1	26.00	01:33.706
MSC Cruises Japanese GP	Sergio Perez	Red Bull Racing Honda RBPT	2	18.00	01:33.945
MSC Cruises Japanese GP	Carlos Sainz	Ferrari	3	15.00	01:33.841
MSC Cruises Japanese GP	Charles Leclerc	Ferrari	4	12.00	01:35.044
MSC Cruises Japanese GP	Lando Norris	McLaren Mercedes	5	10.00	01:35.186
MSC Cruises Japanese GP	Fernando Alonso	Aston Martin Aramco Mercedes	6	8.00	01:34.726
MSC Cruises Japanese GP	George Russell	Mercedes	7	6.00	01:34.404
MSC Cruises Japanese GP	Oscar Piastri	McLaren Mercedes	8	4.00	01:34.802
MSC Cruises Japanese GP	Lewis Hamilton	Mercedes	9	2.00	01:33.952
MSC Cruises Japanese GP	Yuki Tsunoda	RB Honda RBPT	10	1.00	01:36.342

Purpose: Makes it easier to analyse race results without joining multiple tables manually, simply select everything from the view.

Structure: the view brings together DriverResultsGP, *Driver*, *Team*, and *GrandPrix* tables, providing a complete race summary. It orders the results by the race date (most recent first) and finishing position (best performances first).

Database connectivity and Python implementation

Database connectivity description

- For this part, I established a connection using mysql-connector-python between Python and MySQL. Once I was connected I executed my predefined SQL queries by executing my python file. This ran SQL commands to retrieve and manipulate data
- I performed various operations, including selecting data, calculating aggregate values, and modifying records through insertion, updating, and deletion. Besides that my python script included my first five queries (the queries I wrote for level 1). These operations allowed me to interact with the database dynamically from a Python environment, ensuring data retrieval and management
- The connection was established by specifying the database credentials, including the hostname (localhost), user (f1_admin), password, and the database name (Nazir_21848838)
- I was able to automate database interactions and validate the functionality of the schema through real-time queries and updates.

Evidence- executing python code and output

```
test@curtinlab: ~/Desktop/DBS/Nazir_Zernish_21848838_DbS2025
test@curtinlab:~/Desktop/DBS/Nazir_Zernish_21848838_DbS2025$ python3 f1_database_operations.py
Connected to MySQL database
Average Ticket Price: 498.333333 USD

Grand Prix races in March:
('Gulf Air Bahrain GP', datetime.date(2024, 3, 2))
('STC Saudi Arabian GP', datetime.date(2024, 3, 9))
('Rolex Australian GP', datetime.date(2024, 3, 24))

Drivers with names starting with 'L' or Italian nationality:
('Lando Norris', 'United Kingdom')
('Lewis Hamilton', 'United Kingdom')
('Lance Stroll', 'Canada')
('Liam Lawson', 'New Zealand')
('Logan Sargeant', 'United States')

Drivers born after 1995 (Uppercase Names):
MAX VERSTAPPEN
LANDO NORRIS
CHARLES LECLERC
OSCAR PIASTRI
GEORGE RUSSELL
PIERRE GASLY
YUKI TSUNODA
LANCE STROLL
ESTEBAN OCON
ALEXANDER ALBON
OLIVER BEARMAN
FRANCO COLAPINTO
GUANYU ZHOU
LIAM LAWSON
LOGAN SARGEANT

Driver Birth Years:
('Fernando Alonso', 1981)
('Lewis Hamilton', 1985)
('Nico Hulkenberg', 1987)
('Daniel Ricciardo', 1989)
('Valtteri Bottas', 1989)
```

5. Discussion and Reflection

The database design process involved multiple adjustments and iterations to make sure it was efficient and consistent. Initially, certain redundant relationships, such as the ParticipatesIn table, were removed to simplify the schema. The Driver and Car entities were merged due to their one-to-one relationship, reducing unnecessary joins in queries.

During implementation, I added constraints such as ON DELETE CASCADE and CHECK conditions to maintain data integrity. Ensuring proper normalization (up to 3NF) prevented data anomalies, while foreign keys improved query performance.

I faced some challenges along the way. My second trigger didn't update the team standings points correctly upon insertion of a new result. This was because the trigger didn't verify if the driver had an assigned team or check the championship ID of the race, and I fixed it by adding NULL checks for the driver's team and properly linking races to their championships in the trigger logic. It retrieved the correct team id for the result inserted for the specific driver.

I also encountered issues running MySQL scripts because of authentication restrictions. Since MySQL on Ubuntu uses auth_socket for root authentication (I used sudo before logging into MySQL), I was unable to connect using my Python script. To resolve this, I created a new MySQL user with the necessary privileges and updated my connection settings in Python. This ensured smooth execution of queries without requiring sudo access.

Overall, I learnt a lot about both Formula 1 and database implementation. Understanding the thought process and work that goes into it, and how teams, drivers and races are structured

helped me make my database more realistic and efficiently. I'm also glad I gained hands on experience with enforcing constraints, writing more advanced SQL queries and troubleshooting all the issues I faced. Connecting MySQL to Python was a new challenge for me, but a very useful tool and helped me figure out how to manage and execute queries programmatically, an insight into real world database interactions.

If I could do the assignment again, I would add a feature that awards an extra point to the driver who sets the fastest lap in a race, which would be accurate to real F1 rules. I also would have spent more time refining the trigger logic for updating team points, ensuring that edge cases, like drivers without teams or races without a championship, are better handled. This would have made the system more efficient and responsive as the data grows.